

Project Design Status Report

Title

RISC-V NeuroCore: Edge Neural Network Inference via Systolic MAC Array

Group Number : 07

1. System Summary (≤ 4 lines)

- What the system does: A bare-metal RISC-V RV32I CPU dispatches 2-layer MNIST MLP inference to a custom 8×8 INT8 Systolic Array coprocessor via memory-mapped I/O at 0xC000_0000. A Python host GUI sends 784 bytes over UART; a fused ReLU+Bias stage activates each layer in-place; a hardware ArgMax unit identifies the predicted digit displayed on the 7-segment display with a 16-LED confidence bar.
 - Current stage: FPGA implementation — bitstream (neurocore.bit) synthesized and programmed on Nexys A7-100T (XC7A100T-CSG324) running at 50 MHz. All RTL modules pass synthesis and route with 0 errors and 0 critical warnings. Timing is clean: WNS = +5.963 ns, WHS = +0.044 ns.
 - Current working capability: End-to-end MNIST inference operational. UART receive path delivers 784 bytes from host GUI → systolic array executes both layers → ArgMax identifies class → 7-segment display shows predicted digit. All Goal System features (2D Systolic Array, UART live input, hardware ArgMax, 5 C workloads) are complete and integrated. Stretch goals (ping-pong BRAM, BFloat16 unit) not started.
-

2. Scope Tracking

Component	Description	Status	Completion Date	Evidence
Minimum System	3-Stage RISC-V Pipeline (RV32I)	Done	Apr 9	RV32I synthesized; firmware.elf executing on FPGA — PC counter activity confirmed via ILA.
	MMIO Address Decoder (0xC000_0000)	Done	Apr 9	top_fpga.v decodes 0xC0 prefix; gated dmem_we/re confirmed in Vivado netlist.
	Vector MAC Coprocessor (4-lane INT8 + ReLU/Bias FSM)	Done	Apr 9	coprocessor_top.v synthesized; systolic_array.v fully wired in; 1 DSP48E1 inferred. Full-system sim completed.
	Fused ReLU + Bias Stage	Done	Apr 9	relu_en CSR per-layer enable/disable; integrated into systolic array PE datapath.
	BRAM Subsystem (weight / input / output / bias)	Done	Apr 9	All 4 BRAMs synthesized; weights.mem / bias.mem loaded. ROM inference confirmed in Vivado.

	CPU Firmware — test_full_inference.c	Done	Apr 14	Compiles for rv32i; dispatches Layer 1 + Layer 2 + inter-layer INT32→INT8 re-quantisation via shift-add.
	Display Output (7-seg + LEDs)	Done	Apr 19	seven_seg_controller.v and led_controller.v synthesize correctly; class_index updates correctly after zero-logit bug fix.
Goal System	2D Systolic Array (8×8 PE grid)	Done	Apr 19	systolic_array.v + pe.v complete; fully instantiated as u_systolic in coprocessor_top.v (line 75). Replaces vec_mac_engine for both layers.
	Hardware ArgMax Unit (tournament reduction)	Done	Apr 14	argmax_unit.v integrated in coprocessor_top.v; argmax_winner_idx correctly updates class_index after final layer.
	UART Receiver for Live Input	Done	Apr 14	uart_rx.v + uart_input_buffer.v synthesized. GUI sends 784 bytes; transfer triggers inference end-to-end.
	5 Compiled C Workloads	Done	Apr 19	test_layer1.c, test_layer2.c, test_full_inference.c, bench_dot.c, bench_perf.c — all compile for rv32i and run on FPGA.
	Host Python GUI (draw_and_send.py)	Done	Apr 14	draw_and_send.py / pc_verify_gui.py: canvas draws digit, crops to 28×28, sends 784 bytes over pyserial at 115200 baud — confirmed operational.
Stretch Goal	Double-Buffered Ping-Pong BRAM	Not Started	—	Not attempted; buffer week Apr 20–26 reserved for this if time permits.
	BFloat16 Arithmetic Unit	Not Started	—	Not attempted.

3. Design Goals Tracking

Metric	Target	Current	Status
Latency	>50× reduction vs SW for FC layer	Hardware coprocessor path operational end-to-end. Cycle-count benchmarks running via bench_perf.c; measured speedup data being captured from perf_counter.v CSR. Final ratio to be confirmed at demo.	Close
Throughput	≥ 8 MAC ops/cycle	8×8 systolic array = 64 MAC ops/cycle peak (64 PEs operating in parallel). Target of 8 exceeded by 8×.	Met
Resource Usage: LUT	≤ 20% (≤ 12,680 LUTs)	~14,288 LUTs — ~22.5% (Vivado synthesis). LUT count increased significantly due to 8×8 PE grid expansion (64 PEs all LUT-implemented). Exceeds 20% target; mitigation: reduce FSM/ pipeline register width post-demo.	Not Met
Resource Usage: FF	≤ 10% (≤ 12,680 FFs)	~5,068 FFs — ~4.00% (Vivado synthesis). Met.	Met
Resource Usage: BRAM	≤ 15% (≤ 20 tiles)	33 RAMB36 + 3 RAMB18 tiles (~34.5 tile-equivalents) — ~25.6%. Exceeds target due to weight BRAM for 784×128 INT8 matrix + 64 KB IMEM. Mitigation: reduce IMEM/DMEM to 16 KB each post-demo.	Not Met
Resource Usage: DSP	≤ 20% (≤ 48 slices)	1 DSP48E1 — 0.42%. Well within target. PEs are LUT-implemented; single DSP48E1 inferred for weight address computation.	Met
Correctness	≥ 90% on 100-image MNIST test set	End-to-end inference operational — zero-logit root cause resolved. Weight BRAM + scaling verified via verify_inference.py / golden_model.py. Accuracy validation in progress; full 100-image sweep to be completed by final demo.	Close

4. Demonstration Plan Status

Component	Planned	Current Status (1–2 lines)
Input	Live hand-drawn digits transmitted from host PC via UART (draw_and_send.py / pc_verify_gui.py)	UART path fully operational. pc_verify_gui.py captures digit, crops to 28×28, sends 784 bytes at 115200 baud via pyserial. transfer_complete pulse triggers inference start.
Processing	Full end-to-end 2-layer MLP inference using 2D Systolic Array (8×8 PE grid)	systolic_array.v fully integrated in coprocessor_top.v. RISC-V firmware dispatches Layer 1 (784→128 ReLU) and Layer 2 (128→10) via CSR writes. Inter-layer INT32→INT8 re-quantisation handled by firmware shift-add loop.
Output	7-seg predicted digit + 16-LED confidence bar + UART cycle count report	seven_seg_controller.v displays class_index from argmax_unit. led_controller.v shows confidence bar. perf_counter.v CSR provides cycle count; UART TX reports predicted digit and per-layer hw-vs-sw cycle table.
Accuracy	≥ 90% on 100-image MNIST test set; running accuracy printed to UART terminal	verify_inference.py + golden_model.py validate weights and INT8 quantization match. Full 100-image sweep to be run before final demo to confirm ≥ 90% accuracy.

5. Gantt Chart

[illegible]

Activity	Owner	March										April									
Implement uart_bram_writer.v (byte→BRAM packer)	Shubham																				
Modify top_fpga.v: MMIO decoder + clock divider	Avinash																				
Implement argmax_tree.v (4-stage pipelined)	Ved																				
Python: quantize_weights.py + export .mem + weights.h	Samvit																				
Python: generate_test_vectors.py + numpy_golden_model	Samvit																				
RISC-V C: full_inference.c + inter-layer requantisation	Samvit																				
Host GUI: host_gui.py (canvas + pyserial UART send)	Shubham																				
XDC constraints + FPGA synthesis + bitstream generation	All																				
Testbench tb_vec_mac_engine.v (cocotb cosim)	Varun+Ved																				
Debug: trace inference pipeline with ILA / simulation	All																				
Fix: zero-logit root cause (weight BRAM / scaling)	Varun+Avinash																				
Implement pe.v + systolic_array.v integration	Varun																				
RISC-V C: dot_product_bench + cycle_counter_bench	Samvit																				
FPGA demo: end-to-end UART digit → 7-seg (Apr 19)	All																				
Stretch: double-buffer ping-pong BRAM	Avinash																				
Stretch: BFloat16 MAC unit	Varun																				
Final report writing + timing diagram	All																				
Demo video recording (≤4 min)	All																				
Final demo preparation + rehearsal	All																				

6. Risk Tracking

Risk	Status	Impact	Owner	Mitigation Plan
Zero-logit inference output — class_index always 0 despite UART transfer completing	Resolved	High	Varun + Avinash	Root cause identified: weight BRAM row-major packing mismatch + L1_MULT scaling. Fixed via verify_inference.py. End-to-end inference now produces correct class.
BRAM utilization 25.6% exceeds 15% target — caused by 64 KB IMEM + DMEM allocation	Ongoing	Medium	Avinash	Reduce IMEM/DMEM from 64 KB to 16 KB each post-demo; firmware footprint small so this recovers ~24 RAMB36 tiles and brings BRAM utilization within target.
Timing violations (combinational loops) flagged in previous Vivado report	Resolved	High	Avinash + Ved	Registered pipeline stages inserted at loop paths. Post-route timing now clean: WNS = +5.963 ns, WHS = +0.044 ns, 0 critical warnings, 0 errors.
ArgMax 4-stage pipeline latency — logit_regs must be stable ≥ 4 cycles before class_index is valid	Resolved	Medium	Ved	4-cycle post-DONE delay implemented in firmware; argmax_valid flag asserts 4 cycles after cop_done. class_index updates reliably.

7. Per-Person Contribution

Member	Planned	Status	Completion Date
Avinash	MMIO address decoder + top_fpga.v integration	Done	Completed Apr 9
	weight / input / output / bias BRAM modules	Done	Completed Apr 9
	coprocessor_top.v FSM + CSR register file	Done	Completed Apr 14
	Fix: combinational loops + registered pipeline stages (WNS clean)	Done	Completed Apr 17
Varun	vec_mac_engine.v (4-lane INT8 FSM + ReLU/Bias)	Done	Completed Apr 9
	pe.v + systolic_array.v (8x8 PE grid, wired into coprocessor_top.v)	Done	Completed Apr 19
	Debug: zero-logit root cause + weight BRAM fix	Done	Completed Apr 17
Samvit	quantize.py — training + INT8 export + L1_MULT calibration	Done	Completed Apr 9
	test_full_inference.c — 2-layer firmware + re-quantisation	Done	Completed Apr 14
	bench_dot.c + bench_perf.c (cycle-count benchmarks)	Done	Completed Apr 19
	golden_model.py + generate_test_vectors.py	Done	Completed Apr 9
	100-image MNIST accuracy sweep + validation report	WIP	Est. Apr 24
Shubham	uart_rx.v (8N1 FSM, CLKS_PER_BIT=434 @ 50 MHz)	Done	Completed Apr 9
	uart_input_buffer.v (byte-packing + transfer_complete)	Done	Completed Apr 14
	draw_and_send.py + pc_verify_gui.py (Tkinter canvas + pyserial 115200)	Done	Completed Apr 14
	led_controller.v (16-LED confidence bar)	Done	Completed Apr 14
Ved	argmax_unit.v (tournament reduction, integrated + working)	Done	Completed Apr 14
	seven_seg_controller.v (8-digit mux, 50 kHz refresh)	Done	Completed Apr 17
	perf_counter.v + ILA probe setup	Done	Completed Apr 17
	tb_coprocessor.v + tb_systolic.v + tb_systolic_corrected.v cosim	Done	Completed Apr 19

8. GitHub Snapshot

- Repo link: <https://github.com/varunbatra1410/RISC-V-Edge-AI-Accelerator>

Member	Commits (count)
Avinash	22
Samvit	35
Shubham	24
Varun	22
Ved	20
Total	123

Key Commits (with 2-3 line description)

Commit Title	Description
feat: pe.v + systolic_array.v — 8×8 PE grid integration	Implements 8×8 Processing Element grid with pipelined INT8 MAC, fused bias add, and ReLU. Fully instantiated as u_systolic in coprocessor_top.v, replacing vec_mac_engine for both inference layers.
fix: weight BRAM packing + L1_MULT scaling — zero-logit resolved	Row-major 8-lane weight interleaving corrected in quantize.py to match systolic PE read order. L1_MULT re-calibrated; output BRAM verified via verify_inference.py to produce non-zero logits for all 10 classes.
fix: timing loops — registered pipeline stage insertion	Identified 24 combinational loops from Vivado timing report; inserted registered pipeline stages at each. Post-route timing clean: WNS = +5.963 ns, WHS = +0.044 ns, 0 critical warnings.
feat: uart_input_buffer.v — 784-byte frame sync + inference trigger	Replaces uart_bram_writer; packs serial bytes into 32-bit words with 200 ms idle-timeout resync. transfer_complete now directly triggers coprocessor start CSR pulse, enabling fully automated inference on UART receive.
feat: bench_perf.c + perf_counter.v — hw vs sw cycle reporting	bench_perf.c runs dot-product workloads on both RISC-V software path and coprocessor, reads cycle_count CSR from perf_counter.v, and transmits a per-layer hw-vs-sw cycle table over UART TX.

9. Changes Based on TA Inputs

Feedback / Input	Original	Current	Change
Upgrade coprocessor to 2D Systolic Array for better throughput demonstration	vec_mac_engine.v (4-lane 1D engine); systolic_array.v compiled but not wired in	systolic_array.v fully instantiated as u_systolic in coprocessor_top.v (line 75); 8×8 PE grid active for both inference layers.	Completed — 2D Systolic Array live on FPGA; throughput upgraded from 4 to 64 MAC ops/cycle.
Ensure accuracy validation is demonstrated beyond a single example	Single-digit live UART demo planned; no batch accuracy reporting	pc_verify_gui.py + verify_inference.py support sequential 20-image batch; UART terminal prints running accuracy percentage. Full 100-image sweep in progress.	In Progress — batch validation framework built; 100-image sweep to complete by Apr 24.
